

AI Agent 瀏覽器操作與即時除錯工具 — 專案計畫書

專案代號:Agent Eye (暫定)

文件版本:v1.0

日期:2026-07-18

一、專案背景與動機

近期 Google Antigravity 展示了一種新的 AI 輔助開發模式:agent 不只寫程式碼,還能自主啟動前後端服務、開啟瀏覽器實際操作應用程式、透過視覺畫面與 console/network log 判斷是否有 bug,並自行修正、重測,整個過程使用者可即時觀察。

目前 Claude Code 等主流 coding agent 缺乏這種「眼睛」與「手」——它們能寫程式碼,但無法親自驗證程式碼在瀏覽器中的實際行為。本專案目標是打造一套 **VS Code Extension + MCP Server**,將這項能力開放給 Claude Code(以及其他支援 MCP 協定的 agent,如 Cursor、Copilot),讓使用者在 VS Code 中就能享有等同 Antigravity 的「自主開發—測試—除錯」迴圈。

二、專案目標

核心目標

1. 讓 AI agent (以 Claude Code 為優先支援對象) 能夠自主啟動專案的前端 / 後端開發伺服器
2. 讓 agent 能開啟一個**使用者看得到的**瀏覽器視窗，親自操作、測試應用程式
3. 讓 agent 能讀取瀏覽器 console 錯誤、network 請求、畫面截圖，作為除錯依據
4. 讓使用者能在 VS Code 內即時看到 agent 的操作過程與產出 (截圖、log、操作紀錄)，建立信任感
5. 整個能力以標準 **MCP Server** 形式提供，確保不只 Claude Code，未來任何 MCP-aware 工具都能直接使用

非目標 (Out of Scope，至少在 v1 不做)

- 不重新造一個 IDE (不對標「取代 VS Code」，而是作為 VS Code extension 附加能力)
- 不做多 agent 並行協作 / Manager View (Antigravity 的多 agent 派工机制，留待 v2 以後評估)
- 不做雲端代管的 headless agent 執行環境，v1 僅支援本機執行

三、技術架構設計

採用**兩層式架構**，將「能力」與「體驗」分離：

VS Code Extension (使用者體感層)

- 一鍵安裝 / 啟動 MCP Server

- 自動寫入 .mcp.json，讓 Claude Code 自動發現工具

- Webview 側邊欄：即時顯示 agent 截圖 / log / 操作步驟

- Terminal 整合：管理前後端 dev server 生命週期

- 設定介面：允許使用者調整權限（如自動核准的動作範圍）

呼叫 / 讀取產出檔案

MCP Server (能力核心 · 獨立 Node.js process)

- Browser 控制模組 (基於 Playwright · 非 headless)
navigate / click / type / screenshot /
get_console_logs / get_network_requests / evaluate
- Dev Server 管理模組
start_dev_server / stop_dev_server /
get_dev_server_logs / restart_on_crash
- Artifacts 輸出模組
每個動作寫入 .agent-artifacts/*.json 供 UI 層讀取

設計理由：MCP Server 是 Claude Code 真正能呼叫的介面，VS Code Extension API 本身無法被外部 CLI 呼叫。因此核心能力必須以 MCP 協定暴露，VS Code Extension 只負責安裝便利性與視覺化。

技術選型

項目	選擇	理由
瀏覽器自動化	Playwright	支援 <code>launchPersistentContext</code> 開啟真實可見視窗、內建 console/network 監聽、截圖錄影能力完整
MCP Server 基礎	參考 / Fork chrome-devtools-mcp (Google 官方開源)	已實作 CDP 連線、截圖、Lighthouse 效能分析等，避免重工
語言	TypeScript	VS Code Extension 與 MCP SDK 官方皆以 TS 為主，型別安全
Extension UI	VS Code Webview API	用於顯示 Artifacts (截圖、log 時間軸)
程序間通訊	本機檔案系統 (<code>.agent-artifacts/</code>) + 可選 WebSocket	檔案系統最簡單可靠，WebSocket 用於即時性要求較高的畫面更新

四、功能規格清單

MVP (v1.0) 必備功能

- ☐ MCP Tool : `browser_navigate(url)`
- ☐ MCP Tool : `browser_click(selector) / browser_type(selector, text)`
- ☐ MCP Tool : `browser_screenshot()`
- ☐ MCP Tool : `browser_get_console_logs()`
- ☐ MCP Tool : `browser_get_network_requests()`

- ❑ MCP Tool：`start_dev_server(id, command, cwd) / get_dev_server_logs(id) / stop_dev_server(id)`
- ❑ 瀏覽器以非 headless 模式開啟，使用者可即時看到操作
- ❑ VS Code 側邊欄 Webview：顯示最近 N 筆操作的截圖與說明
- ❑ 一鍵指令：Agent Eye: Setup for Claude Code，自動寫入 `.mcp.json`
- ❑ 基本錯誤處理：dev server crash 時通知 agent 並附上 log

v1.1 增強功能

- ❑ 操作錄影 (非只有截圖，串接 CDP `Page.screencastFrame`)
- ❑ Artifacts 時間軸 UI (可回放整個 agent session)
- ❑ 遠端環境支援 (Codespaces / SSH remote：本機彈窗不可行時，改用截圖串流至 Webview)
- ❑ 權限控管：哪些動作需要使用者手動核准 (例如送出表單、刪除操作)

v2.0 展望功能

- ❑ 多 agent / 多瀏覽器分頁並行測試
- ❑ 支援 Firefox / WebKit (跨瀏覽器測試)
- ❑ 與 CI 整合，將同一套 MCP 工具用於自動化回歸測試

五、開發階段與時程規劃

以個人 / 小團隊開發、每週投入約 10–15 小時估算：

階段	內容	預 估 時 程
Phase 0：技術驗證	驗證 chrome-devtools-mcp 能否被 Claude Code 正常呼叫；手動測試 Playwright 非 headless 視窗彈出效果	3–5 天
Phase 1：MCP Server MVP	實作 browser_* 工具集；實作 dev server 管理工具；本機 CLI 測試 (不含 VS Code UI)	1.5– 2 週
Phase 2：VS Code Extension 外殼	建立 extension 專案骨架；實作一鍵設定指令 (寫入 <code>.mcp.json</code>) ；Terminal 整合	1 週
Phase 3：Artifacts 視覺化 UI	Webview 面板；檔案監聽機制；截圖/log 時間軸呈現	1– 1.5 週
Phase 4：整合測試	用真實專案 (如一個簡單的 React + Express 專案) 跑完整流程：agent 寫 code → 啟動服務 → 開瀏覽器測試 → 抓 bug → 修正	1 週
Phase 5：文件與發布	README、使用教學、發布至 VS Code Marketplace (Open VSX 亦可考慮)	3–5 天

總計預估：約 6–8 週 (單人業餘開發；全職投入可壓縮至 3–4 週)

六、風險評估與應對

風險	影響	應對策略
MCP 協定或 Claude Code 的 MCP 支援介面變動	中 ～ 高	鎖定官方 MCP SDK 版本，關注 Anthropic / Claude Code 更新日誌，預留相容性測試時間
Playwright 非 headless 視窗在遠端開發環境 (SSH/Codespaces) 無法彈出	中	v1 先明確支援本機開發情境，v1.1 再補截圖串流方案
Agent 濫用瀏覽器控制權 (例如誤操作真實帳號、送出真實表單)	高	預設僅允許操作本機 localhost 網域；重要動作 (送出/刪除) 需使用者手動核准
效能問題：截圖/log 頻繁寫入拖慢 agent 迴圈	低 ～ 中	限制截圖頻率、log 長度上限，超過則截斷並提示 agent
與現有 chrome-devtools-mcp 功能重疊，重工浪費	中	Phase 0 先完整測試官方套件邊界，只補「它沒有」的部分 (如 dev server 管理、VS Code 內嵌 UI)

七、成功指標 (驗收標準)

- 在一個真實的全端範例專案中，能讓 Claude Code 完成以下無人工介入的完整迴圈：
啟動後端 → 啟動前端 → 開瀏覽器導覽到頁面 → 點擊測試某功能 → 偵測到 console 錯誤 → 修改程式碼 → 重新整理驗證修復
- 使用者能在 VS Code 側邊欄看到至少：目前執行到哪個步驟、最新一張截圖、最近的錯誤 log
- 從安裝 extension 到 Claude Code 能成功呼叫第一個 browser 工具，整體設定時間 < 5 分鐘
- 瀏覽器視窗彈出後，操作延遲 (agent 下指令到畫面反應) 在可接受範圍內 (主觀測試，目標 < 1 秒)

八、參考資料

- Google Antigravity 官方部落格：<https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/>
- Chrome DevTools for agents (開源 MCP Server)：<https://github.com/ChromeDevTools/chrome-devtools-mcp>
- Chrome DevTools for agents 官方文件：<https://developer.chrome.com/docs/devtools/agents>
- Model Context Protocol 規格：<https://modelcontextprotocol.io>
- Playwright 官方文件 (launchPersistentContext / CDP 整合)：<https://playwright.dev>

九、下一步行動

- ☐ Phase 0 技術驗證：手動安裝 chrome-devtools-mcp，測試能否被 Claude Code 呼叫並成功操作瀏覽器

- ☐ 建立專案 monorepo 骨架 (`packages/mcp-server` + `packages/vscode-extension`)
- ☐ 撰寫第一版 MCP tool schema 並跑通最小可行流程 (`navigate` + `screenshot`)